1. Preorder of

preorder: node → left → right

6 17 9 4 22 16 12

↑ E

2. graph, 2d array adjacency matrix, best for lots of edges ($E \approx V^2$)

$\frac{E}{V^2}$ ← slots

A. $\frac{1200}{1000000} = 0.12\%$

B. $\frac{4000}{10000} = 40\%$

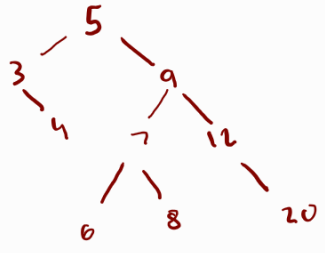
C. $\frac{10000}{1000000} = 1\%$

D. $\frac{20}{100} = 20\%$

3. worst hash search looks at everything, $O(N)$

4. 1800 license plates, speed of search most important open addressing hash table average search is $O(1)$ (good hash function), so C.

5. which won't produce?



A.

B.

C.

A. 5 3 4 9 12 7 8 6 20

B. 5, 9, 3, 7, 6, 8, 4, 12, 20

C. 5 9 7 8 6 12, 20, 3, 4

D. 5 9 7 3 8 12 6 4 20

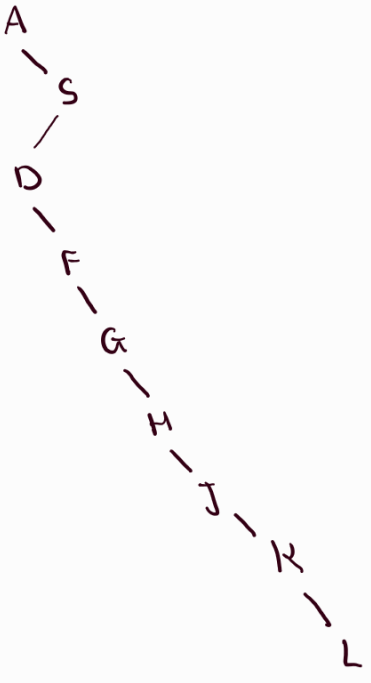
E. 5 9 3 6 7 8 4 12 20

D.

E.

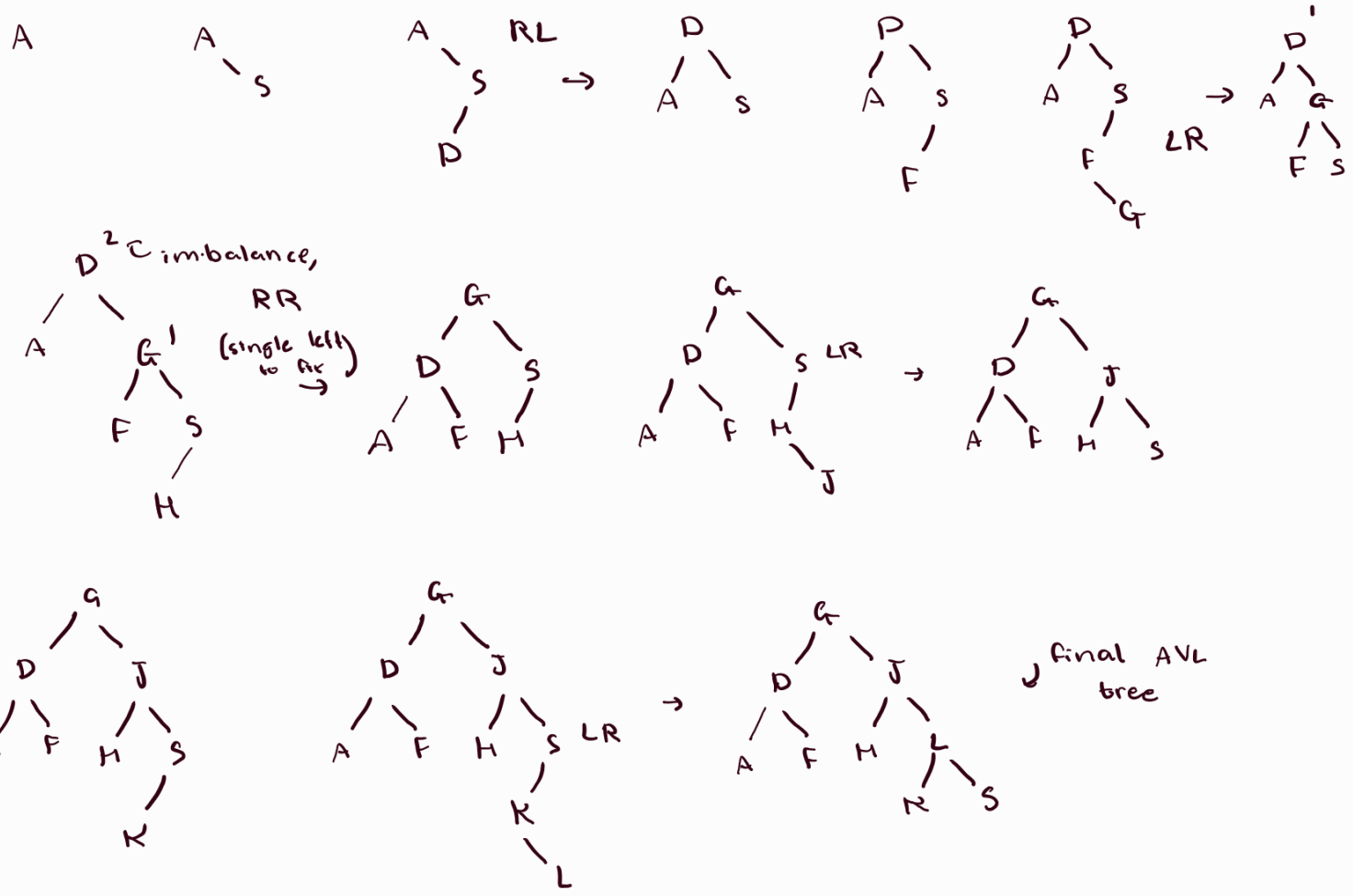
11-1. a) ASDFGHJKL BST

A-1 N-14
B-2 O-15
C-3 P-16
D-4 Q-17
E-5 R-18
F-6 S-19
G-7 T-20
H-8 U-21
I-9 V-22
J-10 W-23
K-11 X-24
L-12 Y-25
M-13 Z-26



(not exactly a great BST)

II-1 b) ASDFGHJKL AVL Tree



II-2 Hashing , 15 17 8 23 3 5

a.

0	
1	15
2	8
3	17
4	23
5	3
6	5

$h(x) = x \% 7$

linear probing

$h(15) = 15 \% 7 = 1$

$h(17) = 17 \% 7 = 3$

$h(8) = 8 \% 7 = 1$

↳ $h(8) = (1+1) \% 7 = 2$

$h(23) = 23 \% 7 = 2$

↳ $h(23) = (2+1) \% 7 = 3$

↳ $h(23) = (2+2) \% 7 = 4$

$h(3) = 3 \% 7 = 3$

↳ $h(3) = (3+1) \% 7 = 4$

↳ $h(3) = (3+2) \% 7 = 5$

$h(5) = 5 \% 7 = 5$

↳ $h(5) = (5+1) \% 7 = 6$

b.

0	
1	15
2	8
3	17
4	3
5	5
6	23

$h(x) = x \% 7$; quadratic probing

$h(15) = 15 \% 7 = 1$

$h(17) = 17 \% 7 = 3$

$h(8) = 8 \% 7 = 1$

↳ $h(8) = (1+1^2) \% 7 = 2$

$h(23) = 23 \% 7 = 2$

↳ $h(23) = (2+1^2) \% 7 = 3$

↳ $h(23) = (2+2^2) \% 7 = 6$

$h(3) = 3 \% 7 = 3$

↳ $h(3) = (3+1^2) \% 7 = 4$

$h(5) = 5 \% 7 = 5$

c.

0	
1	15
2	23
3	17
4	3
5	8
6	5

$h(x) = x \% 7$, double hashing, $h_2(x) = x / 7 + 1$

$h(15) = 15 \% 7 = 1$

$h(17) = 17 \% 7 = 3$

$h(8) = 8 \% 7 = 1$

↳ $h(8) = (1 + h_2(8)) \% 7$

$= (1 + 2) \% 7 = 3$

↳ $h(8) = (1 + 2 * h(8)) \% 7$

$= (1 + 4) \% 7 = 5$

$h(23) = 23 \% 7 = 2$

$h(3) = 3 \% 7 = 3$

↳ $h(3) = (3 + 1 * h_2(3)) \% 7$

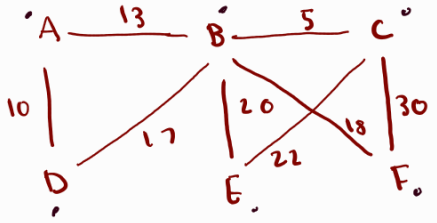
$= (3 + 1) \% 7 = 4$

$h(5) = 5 \% 7 = 5$

↳ $h(5) = (5 + 1 * h_2(5)) \% 7$

$= (5 + 1) \% 7 = 6$

11-6 Graph Algorithms



smallest edge first

CBDAFE

a) Depth First Traversal, start at C

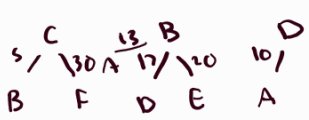
C → B → D → A → F → E

no new edges, goes back to B

no new edges, goes back to B

b) Breadth First Traversal, start at C

C → B → F → A → D → E

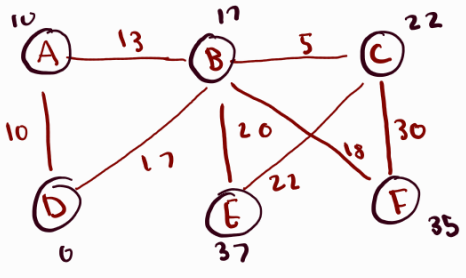


CBF ADE

from C, from B

c) Dijkstra's algorithm to find shortest path from D to E

start at D, pick local shortest



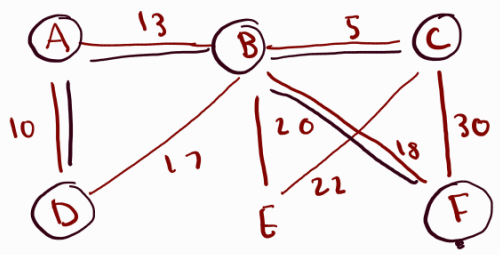
A	B	C	D	E	F	
∞	∞	∞	0	∞	∞	D
10	17	∞	0	∞	∞	
10	~~23~~	∞	0	∞	∞	A
10	17	22	0	37	35	B
10	17	22	0	~~45~~	~~52~~	C
10	17	22	0	37	35	F
10	17	22	0	37	35	E

lengths of shortest paths

Smallest / Shortest path from D to E is D → B → E (37)

D → A → B → C → F → E

II-4 Graph Algorithms



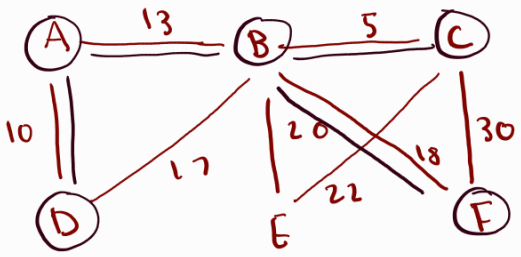
a) Prim's algorithm, start at F (I'm not keeping them in min heap order)

(F, B, 18) (F, C, 30) (B, C, 5) (B, A, 13) (B, D, 17)
(B, E, 20) (A, D, 10)

1st 4 edges — means processed

1 2 3 4

First 4 edges picked: (F, B), (B, C), (B, A), (A, D)

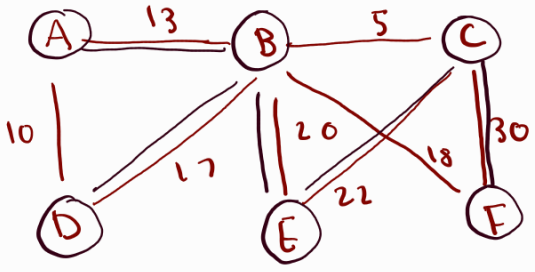


b) Kruskal's, first 4 edges

(B, C, 5) (A, D, 10) (A, B, 13) (B, D, 17) (B, F, 18)
(B, E, 20) (E, C, 22) (C, F, 30)

2 2 3 4

∴ First 4 edges are (B, C), (A, D), (A, B), (B, F)



c) non minimal spanning tree

I'll do the opposite of Kruskals

(C, F), (C, E), (B, E), (B, D), (B, A)

spanning tree - all vertices, no cycles

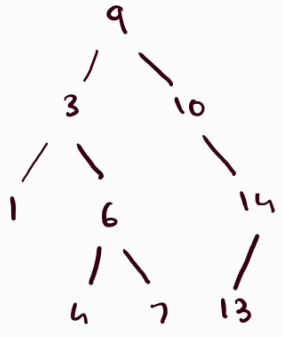
no question number: Draw Binary tree given by

index	0	1	2	3	4	5	6	7	8	9	10	11	12	13
item	9	3	10	1	6		14			4	7			13
level	0	1		2				3						

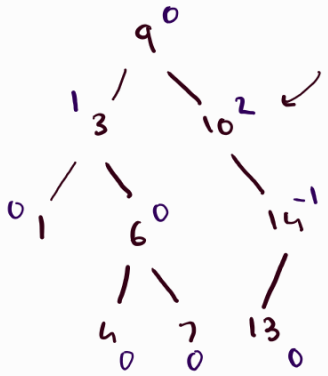
left child: $2i+1$

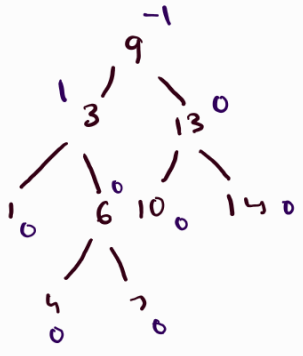
right child: $2i+2$

0 - root



to AVL tree

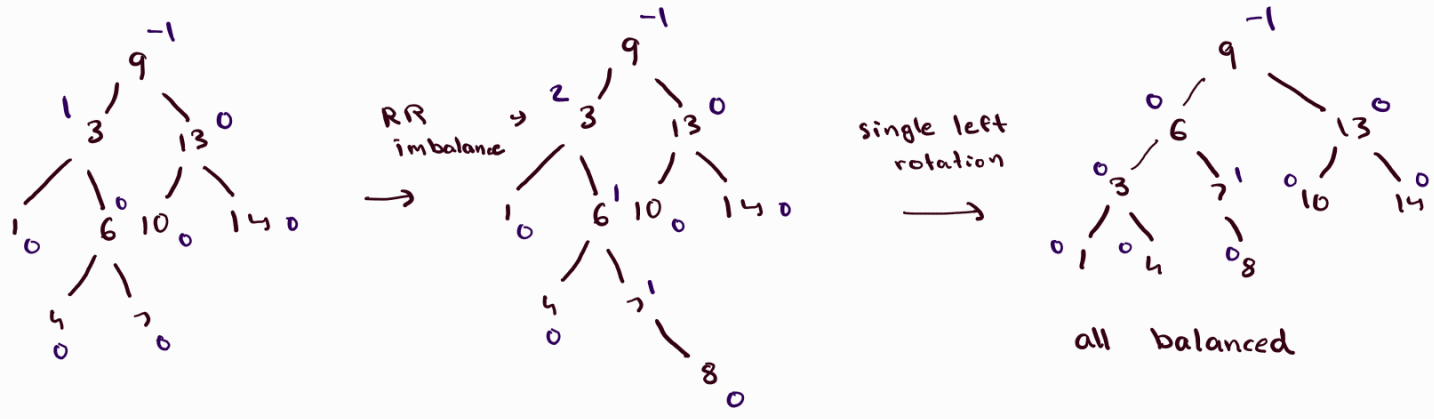




all balanced

0	1	2	3	4	5	6	7	8	9	10	11	12	13
9	3	13	1	6	10	14			4	7			

Insert node (8) into AVL Tree



top-down quick sort, last element is pivot

Cat Bit Hat Mat Rat Bat Hot Bot Sit ← stays at end

| < < < < < < < | pivot

| > < > > > < > | pivot

Bit Cat

Bat

Bot

Cat

Hat

Bit Bat Bot Mat Rat Cat Hot Hat Sit

| > | pivot | | > > < > pivot |

Bat Bit Bot Cat Rat Mat Hot Hat

Cat Hat Mat Hot Rat

| pivot |

Bat Bit Bot Cat Hat Mat Hot Rat Sit

| < < pivot |

| > pivot | Rat

Hot Mat

Bat Bit Bot Cat Hat Hot Mat Rat Sit

↑ final sorted array

Supplemental

1. BST, no dupes, how delete node with two children, left subtree

we look for the largest node in the left subtree - in order predecessor

a)

2.
```
public static int recurse(int a, int b) {
    if (a % b == 2){
        return a;
    } else {
        return recurse(a+b, a-b);
    }
}
```

recurse (7, 2)

7 % 2 = 1 ∴ recurse (7, 2)

↳ recurse($\overset{9}{7+2}$, $\overset{5}{7-2}$)

9 % 5 = 4

↳ recurse ($\overset{14}{9+5}$, $\overset{4}{9-5}$)

14 % 4 = 2 ∴

↳ return 14

∴ answer is 14 (D)

3. sorting, does not require $O(n^2)$ worst

A. Insertion $O(n^2)$ if sorted in reverse order

B. selection always $O(n^2)$

(C) heap always $O(n \log n)$ (Build heap, $O(\log n)$ insertion guaranteed)

D. quick $O(n^2)$ if sorted, pivot first or last